

VSF Intern · Track MLE · 06/10/2026

Tổng kết 3 tuần thực tập lần I

Người trình bày: **Trần Chí Tâm**

Nội dung trình bày

1

Giới thiệu

Mục tiêu buổi trình bày · hai dự án

2

MovieLens Recommendation API

3

GeoLife Home / Office

4

Kế hoạch sắp tới

Việc tiếp theo của dự án GeoLife

5

Tổng kết

Tóm lại toàn bộ buổi trình bày

1. Giới thiệu

Mục tiêu của buổi trình bày và hai dự án đã thực hiện.

VSF Intern · Track MLE

1. Giới thiệu

Mục tiêu buổi trình bày và hai dự án

Mục tiêu: tổng hợp lại các công việc đã làm cũng như những gì đã học được.

DỰ ÁN 1

MovieLens Recommendation API

Hệ gợi ý phim bằng SVD, đóng gói thành dịch vụ và triển khai trên GCP Cloud Run.

Trọng tâm học: tìm hiểu ba chiến lược triển khai (shadow, canary, blue-green) và triển khai thực tế canary 90 / 10 trên Cloud Run.

100,000

lượt đánh giá
(MovieLens 100K)

943

user

1,682

phim

DỰ ÁN 2

GeoLife Home / Office

Suy ra nhà, cơ quan và địa điểm hay đến từ quỹ đạo GPS, phục vụ qua API.

Trọng tâm học: khoa học dữ liệu trên tín hiệu vị trí: làm sạch, tìm stay-point, kiểm chứng khi không có nhãn.

24.88 M

điểm GPS thô

182

user · 18,670 file .plt

18,597

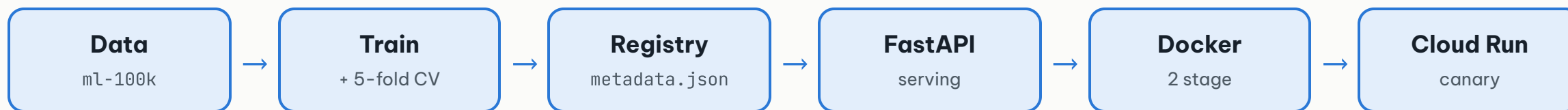
stay-point

2. MovieLens Recommendation API

Hệ gợi ý phim.

Từ mô hình SVD đến dịch vụ gợi ý phim trên Cloud Run

Gợi ý phim bằng SVD (scikit-surprise) trên MovieLens 100K.



v1.0 · production

Train trên toàn bộ dữ liệu · `svd_model_full`

v2.0 · staging

Train trên fold 5 · `svd_model_fold5`

Đánh giá

Cross-validation 5 fold (RMSE / MAE); ablation `n_factors`, `n_epochs`.

Serving

FastAPI + Pydantic, 4 router; spec OpenAPI viết trước khi code.

Đóng gói

Dockerfile 2 stage (builder / runner), docker-compose có healthcheck.

Kiểm thử

Postman ~40 request (404 / 422, user journey), pytest, CI flake8 + pytest.

2. MovieLens Recommendation API (2/4)

Làm được gì?

Kiểm chứng thực tế: cấu hình 90 / 10, đo được 94 / 6

① Cấu hình trên Cloud Run

Name	Traffic	Deployed ↓	Re	Actions
movielens-api-can-1789616600	10%	5 minutes ago		
movielens-api-canary	90%	19 hours ago		
movielens-api-00007-p2c	0%	21 hours ago		
movielens-api-00006-m98	0%	22 hours ago		
movielens-api-00005-r2n	0%	22 hours ago		
movielens-api-00004-g6w	0%	22 hours ago		
movielens-api-00003-x8g	0%	23 hours ago		
movielens-api-00002-bzv	0%	23 hours ago		
movielens-api-00001-ph6	0%	23 hours ago		

Revision mới `movielens-api-can-1789616600` nhận 10%, revision đang chạy nhận 90%; các revision cũ 0%.

② Đo bằng `scripts/test_traffic.py`

```
(movielens) D:\Hoc_voi_cha_hanh\AIInAction\VSF_Intern\MovieLensProject\scripts>python test_traffic.py
🔥 Gửi 100 requests kiểm tra Traffic Splitting: https://movielens-api-366360236110.us-central1.run.app/health

-> Đã gửi 20/100 requests...
-> Đã gửi 40/100 requests...
-> Đã gửi 60/100 requests...
-> Đã gửi 80/100 requests...
-> Đã gửi 100/100 requests...

=====
📊 KẾT QUẢ XÁC THỰC CANARY DEPLOYMENT (90/10)
=====
• Model: svd_model_full      | 94 requests ( 94.0%)
• Model: svd_model_fold5     |  6 requests (  6.0%)
=====
```

Gửi 100 request tới `/health`, đếm theo `model_name`: 94 tới `svd_model_full` (production), 6 tới `svd_model_fold5` (canary).

Với 100 request, số request tới canary dao động quanh 10 với độ lệch chuẩn ≈ 3 ($\sqrt{(100 \times 0.1 \times 0.9)}$); đo được 6 là lệch khoảng 1.3 độ lệch chuẩn, vẫn khớp cấu hình 90 / 10.

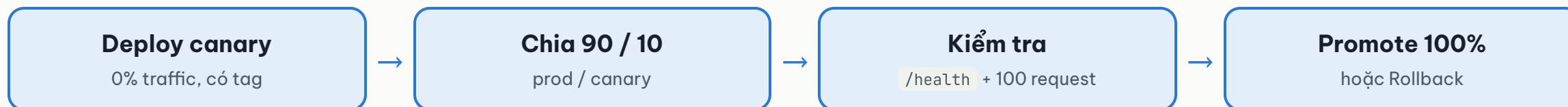
Chọn chiến lược triển khai: vì sao là canary?

CHIẾN LƯỢC	CÁCH HOẠT ĐỘNG	GHI CHÚ
Shadow	Model mới nhận bản sao request, người dùng không thấy kết quả	Không có phản hồi trực tiếp từ người dùng
Canary ✓	Một phần nhỏ người dùng thật dùng model mới	Phù hợp feedback loop
Blue-green	Chuyển 100% traffic một lần	Không thử dần trên người dùng

Lý do chọn canary

Có phản hồi trực tiếp từ người dùng thật để cải thiện model, trong khi phần lớn người dùng vẫn ở model production ổn định. Ba chiến lược được tìm hiểu để so sánh; chỉ canary được triển khai thực tế.

Vận hành canary: triển khai, chia traffic, kiểm chứng



Cơ chế

- Cùng một image, hai revision (`prod-...` / `can-...`).
- Model chọn bằng biến môi trường `MODEL_NAME` :
`svd_model_full` / `svd_model_fold5` .
- `/health` trả về `model_name` để biết request đi tới model nào.

Điều khiển

- GitHub Actions `deploy.yml` chạy tay (`workflow_dispatch`): `deploy-canary`, `set-traffic`, `promote`, `rollback`.
- Mọi action gọi lại `deploy.sh` ; `deploy-canary` = build → deploy 0% → tự chia 90/10.
- Push và pull request chỉ chạy Test & Lint, không tự deploy.

3. GeoLife Home / Office

Suy ra nhà và cơ quan từ quỹ đạo GPS.

Suy luận nhà và cơ quan từ quỹ đạo GPS không nhãn

Từ GPS thô của một người, tìm **Home**, **Office** và các địa điểm hay đến (POI), trả về qua API.



Dữ liệu Microsoft GeoLife **không có nhãn nhà hay cơ quan**.
Nhãn chỉ là phương tiện di chuyển và phủ 20.1% số điểm.

Không có đáp án để đối chiếu, nên mỗi bước làm sạch đều có **notebook bằng chứng**; dòng bị loại không bị xóa âm thầm mà ghi vào **quarantine** kèm lý do.

Quarantine: dòng bị loại được ghi lại kèm lý do

Dữ liệu lỗi không bị xoá âm thầm: mỗi điểm bị loại được ghi vào một file riêng của từng user kèm cột `reason` (tổng 795 điểm).

LÝ DO (REASON)	NGHĨA LÀ	VÌ SAO TÁCH RIÊNG
<code>speed_spike</code>	Điểm mà tốc độ tới và đi đều > 1,100 km/h nhưng bỏ điểm đó thì không	Lỗi GPS đơn lẻ. Chuyển động nhanh kéo dài vẫn được giữ, chỉ điểm nhảy bị loại.
<code>duplicate_rejected_candidate</code>	Cùng một giây có nhiều toạ độ; ứng viên bị loại vì tốc độ tới hai điểm neo quá lớn	Các ứng viên gần như cùng một chỗ (trung vị 1.4 m), nên giữ ứng viên hợp lý nhất và ghi lại số còn lại.
<code>duplicate_no_valid_candidate</code>	Không ứng viên nào hợp lệ	Không có gì đáng tin để giữ, nên loại cả nhóm.
<code>duplicate_invalid_anchors</code>	Hai điểm neo hai bên đã cách nhau > 1,100 km/h	Lỗi nằm ở điểm neo chứ không ở ứng viên; 13/16 nhóm được cứu lại bằng điểm neo mới.
<code>duplicate_ambiguous_candidates</code>	≥ 2 ứng viên hợp lệ nhưng cách nhau > 200 m	Chọn cái nào cũng là đoán và làm đổi stay-point: hai quy tắc chọn bất đồng ở 27/52 nhóm.

Phân loại theo **nguyên nhân** vì mỗi loại cần quyết định khác nhau (loại điểm, loại cả nhóm, hoặc cứu lại) và có thể kiểm tra lại từng trường hợp.

Quarantine bằng hình: điểm nhảy và điểm trùng giây

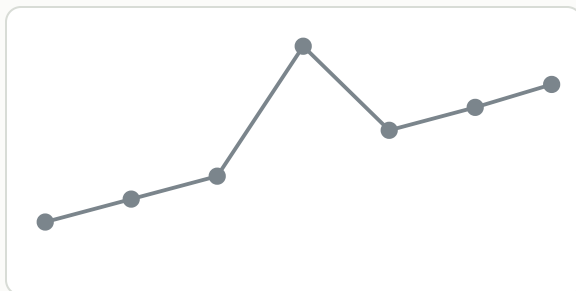
LÝ DO

speed_spike

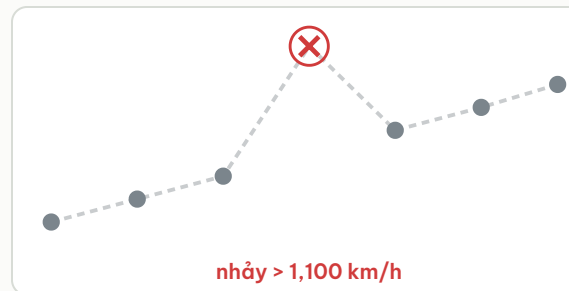
Điểm nhảy đột ngột

Cách hai điểm kế > 1,100 km/h trong khi hai điểm kế nhau thì bình thường: bỏ điểm nhảy.

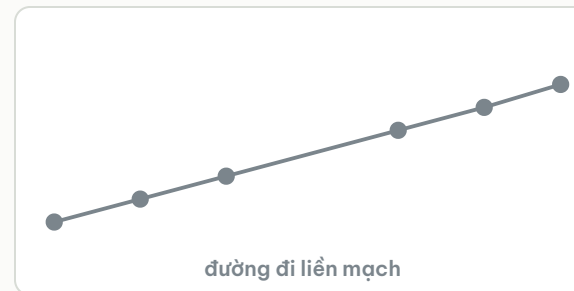
① TRƯỚC KHI LOẠI



② CHỌN ĐIỂM LOẠI (ĐỎ)



③ SAU KHI LOẠI



duplicate_rejected_candidate

Cùng giây, nhiều toạ độ

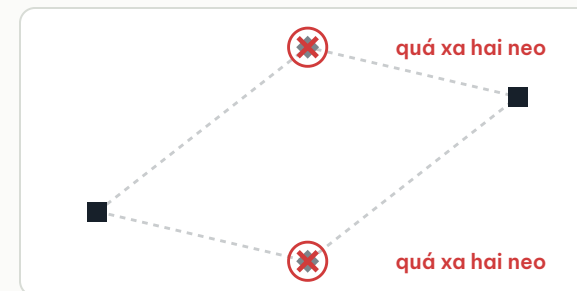
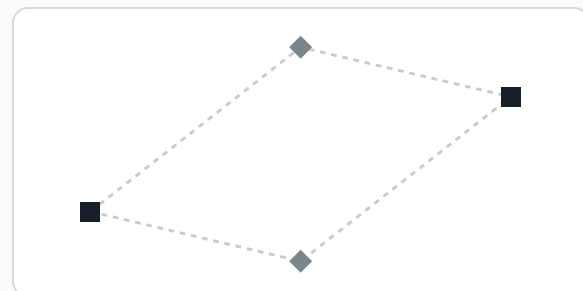
Giữ ứng viên hợp lý nhất; loại ứng viên có tốc độ tới hai điểm neo quá lớn.



duplicate_no_valid_candidate

Không ứng viên nào hợp lệ

Mọi ứng viên đều quá xa hai điểm neo: loại cả nhóm.



■ điểm neo: điểm hợp lệ gần nhất ngay trước và ngay sau · ◆ ứng viên: các toạ độ cùng một giây · ✗ điểm bị đưa vào quarantine · Hình minh hoạ sơ đồ, không phải dữ liệu thật.

Quarantine bằng hình: lỗi do điểm neo và nhiều ứng viên

LÝ DO

① TRƯỚC KHI LOẠI

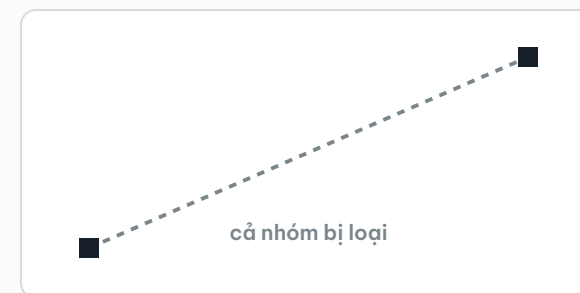
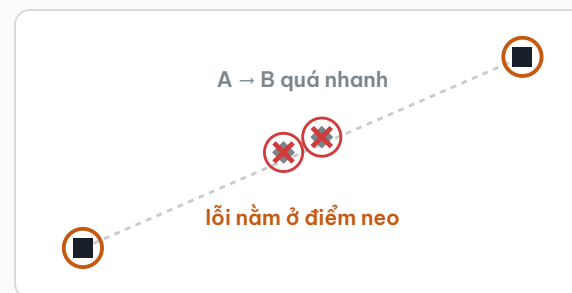
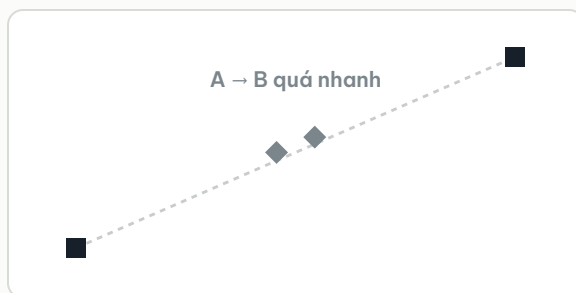
② CHỌN ĐIỂM LOẠI (ĐỎ)

③ SAU KHI LOẠI

duplicate_invalid_anchors

Điểm neo bất thường

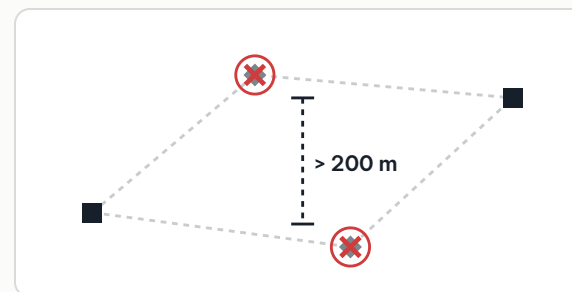
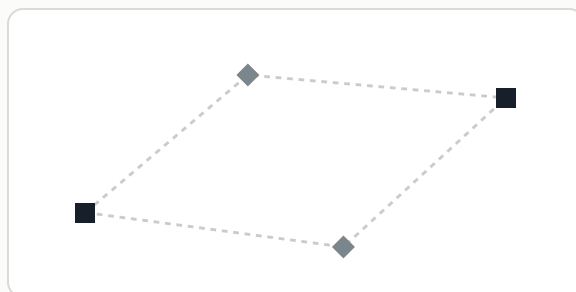
Hai điểm neo đã cách nhau > 1,100 km/h nên không ứng viên nào đúng: loại cả nhóm (13/16 nhóm sau đó được cứu bằng điểm neo mới).



duplicate_ambiguous_candidates

Nhiều ứng viên hợp lệ, cách xa nhau

≥ 2 ứng viên hợp lệ cách nhau > 200 m: chọn cái nào cũng là đoán, nên loại cả nhóm.



■ điểm neo: điểm hợp lệ gần nhất ngay trước và ngay sau • ◆ ứng viên: các toạ độ cùng một giây • ✗ điểm bị đưa vào quarantine • Hình minh họa sơ đồ, không phải dữ liệu thật.

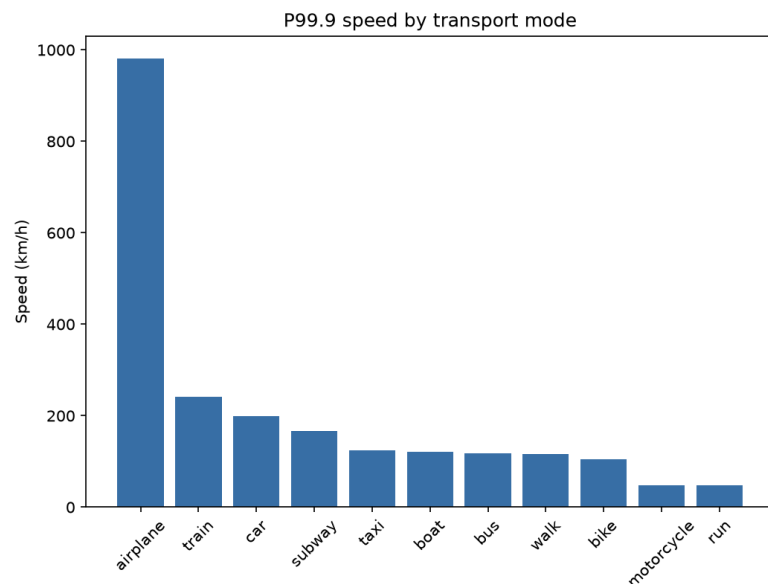
Mỗi ngưỡng xử lý đều có bằng chứng từ dữ liệu

QUYẾT ĐỊNH	BẰNG CHỨNG (DỮ LIỆU THẬT)	KẾT QUẢ
Ngưỡng tốc độ bất khả thi	Điểm thọt nhanh nhất 1,048 km/h; lỗi GPS chậm nhất 1,123 km/h	1,100 km/h
Cắt đoạn khi mất tín hiệu	Khoảng ngắt trong một chuyến: P99.9 = 252 s, P99.99 = 2,170 s; không có điểm gãy tự nhiên, ở 20 phút chỉ 0.02% khoảng ngắt trong chuyến bị cắt nhầm	20 phút (quy ước)
Nổi các file của một user	Tỉ lệ "ghi lại ở cùng chỗ" chạm mức nền 19.7% ở 18 h; số đêm tìm được 395 → 3,340	≤ 18 h
Giờ địa phương	2.61% số điểm ngoài UTC+8; UTC+8 cố định đặt 21.5% hoạt động vào 1–5 h sáng	Theo tọa độ
Bán kính / thời gian stay-point	Ngưỡng phổ biến trong các nghiên cứu trên GeoLife (Li et al. 2008; Zheng et al.), kiểm tra bằng cách xem stay-point trên bản đồ	200 m / 30 phút (tham khảo)

Bằng chứng chi tiết cho từng ngưỡng

Ngưỡng tốc độ bất khả thi

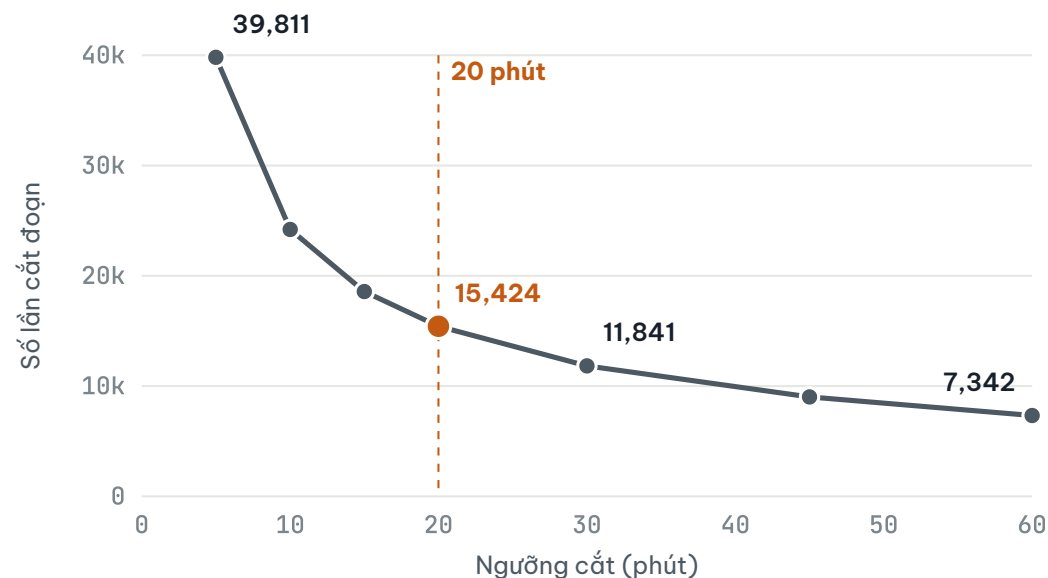
1,100 km/h



Điểm thật nhanh nhất (máy bay) là 1,048 km/h; lỗi GPS chậm nhất là 1,123 km/h. Ngưỡng đặt ở giữa hai mức này.

Cắt đoạn khi mất tín hiệu

20 phút (quy ước)

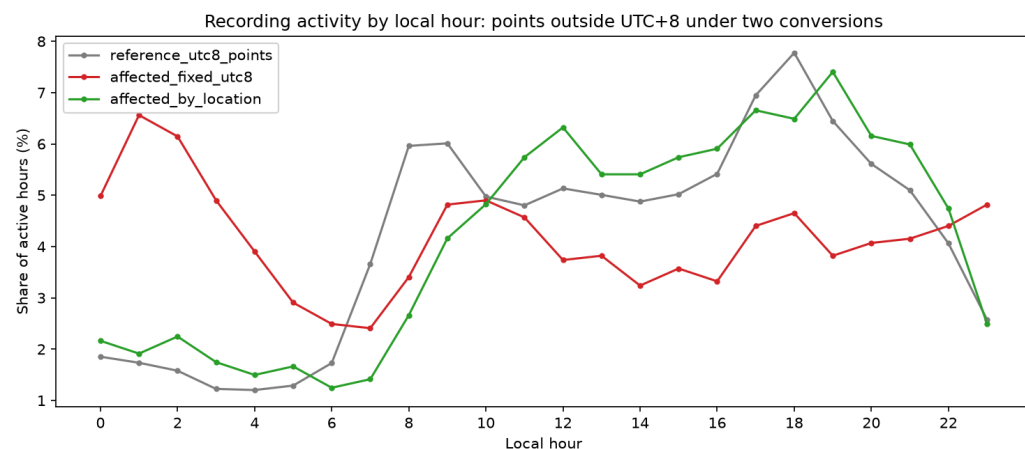


- Trong chuyến, 99.9% khoảng ngắt dưới 252 s; ở 20 phút chỉ 0.02% khoảng ngắt trong chuyến bị cắt nhầm.
- Số lần cắt giảm đều, **không có điểm gãy**: 20 phút là ngưỡng quy ước, không phải tối ưu (30 phút: 0.012%).
- Đã chạy thử** 10 / 20 / 30 phút trên 16 user (3.7 triệu điểm): số đoạn đổi 6,099 / 4,865 / 4,412 nhưng 2,413 stay-point giống hệt nhau.

Bằng chứng chi tiết cho từng ngưỡng

Giờ địa phương

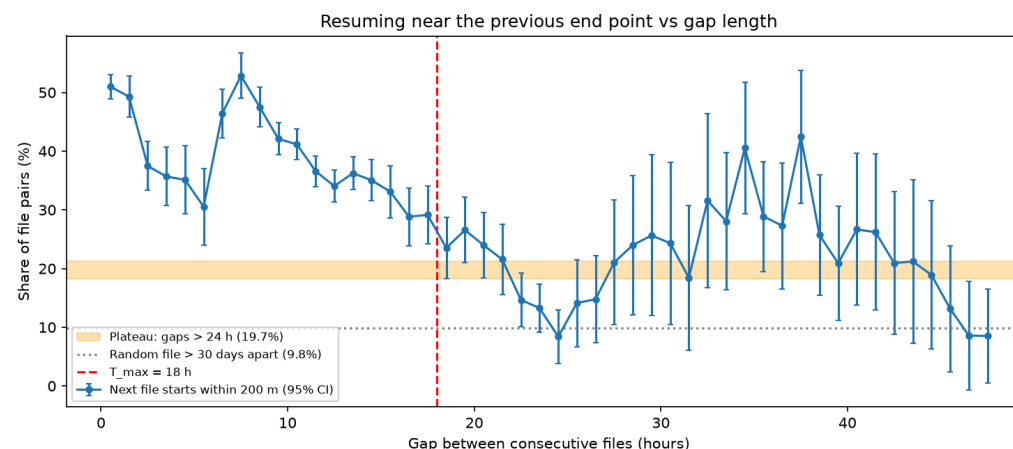
Theo tọa độ



- Vấn đề:** GPS ghi giờ GMT. Cộng cố định 8 giờ (giờ Bắc Kinh) sẽ sai với 2.61% số điểm ở nước ngoài.
- Cách đọc:** trục ngang là giờ trong ngày, trục dọc là tỉ lệ thời gian có ghi GPS. **Xám:** user ở Trung Quốc, nhịp sinh hoạt bình thường (thấp lúc 2–6 h sáng). **Đỏ:** điểm ở nước ngoài nếu cộng cố định 8 giờ. **Xanh:** cùng các điểm đó, đổi theo múi giờ tại tọa độ.
- Kết luận:** đường đỏ có đỉnh lúc 0–2 h sáng (21.5% hoạt động vào 1–5 h), vô lý; đường xanh giống đường xám (7.4%). Phải đổi giờ theo tọa độ, nếu không nhà (tìm theo giờ đêm) sẽ sai.

Nối các file của một user

≤ 18 h



- Vấn đề:** mỗi lần bật/tắt máy là một file. Đêm ở nhà thường nằm **giữa** hai file: xét từng file riêng thì mất đêm, nối tất cả thì ra stay-point dài nhiều năm.
- Cách đọc:** trục ngang là thời gian nghỉ giữa hai file, trục dọc là tỉ lệ file sau bắt đầu trong 200 m quanh chỗ file trước kết thúc (tức là đã ở yên). **Đải cam 19.7%:** mức do thói quen quay lại chỗ quen, đo khi nghỉ > 24 h.
- Kết luận:** nghỉ ngắn thì ~50% bật lại đúng chỗ cũ; đến **18 h** (đường đỏ) tỉ lệ rơi xuống mức thói quen. Nên chỉ nối khi nghỉ ≤ 18 h: số đêm tìm được tăng 395 → 3,340.

Một số phương pháp phân tích và đánh giá đã áp dụng

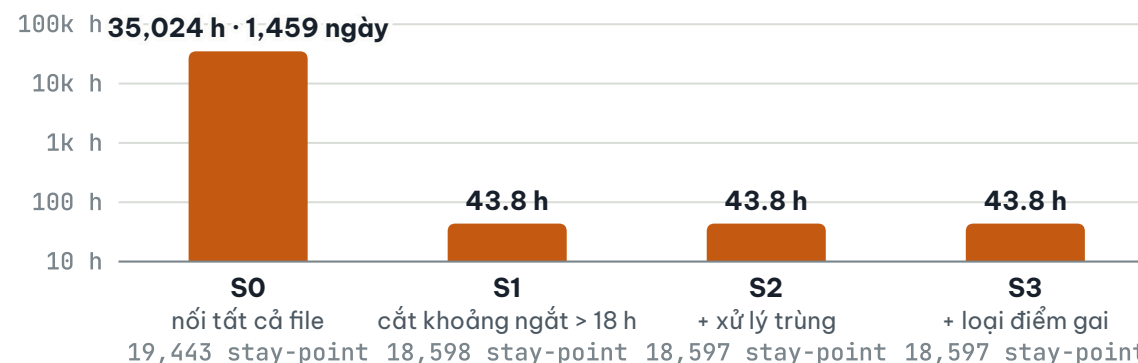
1 Tắt từng bước, xem kết quả đổi bao nhiêu.

Nối mọi file với nhau cho ra một lần ở yên dài 1,459 ngày; chỉ nối khi khoảng ngắt ≤ 18 h thì còn 43.8 giờ. Các bước xử lý lỗi khác gần như không đổi gì (hình trên).

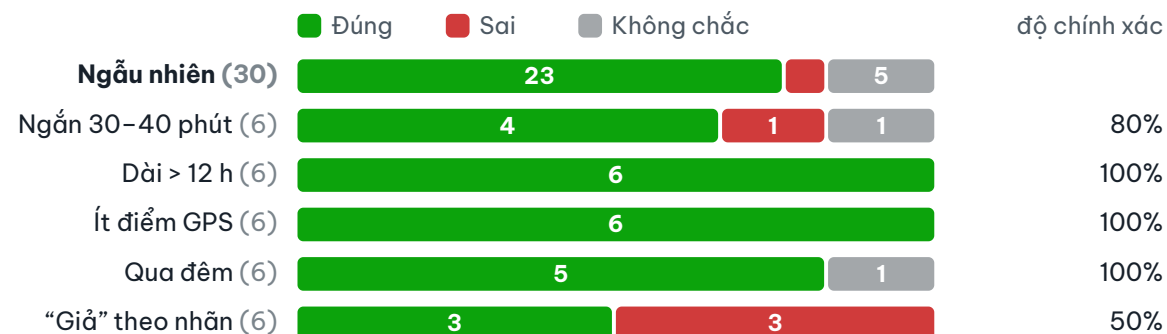
2 Chấm thủ công, không biết nhóm.

Người chấm xem bản đồ 60 lần ở yên và kết luận đúng/sai mà không biết nó thuộc nhóm nào. Lỗi duy nhất tìm thấy là xe đi chậm trên đường lớn (hình dưới).

Tắt từng bước: lần ở yên dài nhất khi bật dần từng bước (giờ, thang log)



Chấm mù 60 stay-point: độ chính xác tính trên các ca đã kết luận (bỏ “không chắc”)



4. Kế hoạch sắp tới

Những việc tiếp theo của dự án GeoLife.

GeoLife

Dự án GeoLife: các bước tiếp theo

1

Tính Home / Office theo thời gian ở lại trong khung giờ

Quy tắc theo giờ đến hiện chỉ tìm được Home cho 130 / 174 user.

2

Bổ sung kết quả "chưa đủ dữ liệu" và đưa cấu hình vào settings

7 user không có stay-point nên classifier cần một kết quả riêng cho trường hợp này.

3

v2: gom địa điểm bằng DBSCAN

Thay cho cách gom theo toạ độ làm tròn của v1.

4

Đánh giá classifier

Gộp các user dùng chung file khi chia train / test để tránh đếm trùng và rò rỉ dữ liệu.

5

Đóng gói và triển khai

Docker và deploy (checkpoint 2), sau đó giám sát (checkpoint 3).

5. Tổng kết

Tóm lại toàn bộ buổi trình bày.

MovieLens · GeoLife

5. Tổng kết

Điều động lại sau 3 tuần

MOVIELENS · TRIỂN KHAI

Đưa model thành dịch vụ chạy thật

- Pipeline hoàn chỉnh: train, registry, FastAPI, Docker, Cloud Run.
- Tìm hiểu ba chiến lược shadow, canary, blue-green; triển khai thực tế canary.
- Kiểm chứng canary: cấu hình 90 / 10, đo được 94 / 6.
- Bài học: revision tự khai báo model; model và dữ liệu đi cùng image; rollback chỉ là đổi traffic.

GEOLIFE · DỮ LIỆU

Từ GPS thô đến kết luận có bằng chứng

- 24.88 M điểm GPS → 18,597 stay-point của 174 user.
- Mỗi quyết định và ngưỡng đều có notebook, số liệu thật.
- Kiểm chứng khi không có nhãn: kiểm tra 9 tính chất bắt buộc và chấm mù một mẫu stay-point.
- Bài học: phải kiểm tra kết quả trên dữ liệu thật, không chỉ kiểm tra code chạy đúng.

Thông điệp chung: một hệ thống ML đáng tin khi cả cách đưa nó ra chạy lẫn dữ liệu đầu vào đều được kiểm chứng bằng đo đạc, không bằng giả định.

Q&A

Câu hỏi và thảo luận.

Thanks for your attention

[MovieLens Recommendation API](#) · [GeoLife Home](#) / [Office](#)